

Numérique Durable (TI616)

Mini-Projet Hackathon : « Le site web le plus Green »

Rapport Final — Partie 1 + Partie 2

Projet : GreenCode Hub

URL : <https://greencode-hub.onrender.com>

GitHub : <https://github.com/YvesDK07/GreenCode-Hub>

Module : TI616 – Numérique Durable

Équipe :

- Yves de KERROS — Back-end (routes API, authentification)
 - Raphael GIRARD — Front-end (pages, composants, CSS)
 - Lestyn JONES — Base de données (modèles, requêtes, migrations)
 - Yohan DE LANDTSHEER — Documentation, tests, déploiement
 - Robinson DIALLO — Front-end complémentaire, wireframes, design sobre
-

Table des matières

Section 1 — Présentation du projet

Section 2 — Architecture et conception

Section 3 — Choix technologiques justifiés

Section 4 — Implémentation

Section 5 — Analyse de l'empreinte carbone

Section 6 — Tests et validation

Section 7 — Organisation de l'équipe et collaboration

Section 8 — Discussion et conclusion

Section 9 — Annexes

Section 1 — Présentation du projet

Proposition de valeur : GreenCode Hub est un site sobre où les développeurs partagent et comparent des snippets de code classés par efficacité énergétique selon leur complexité algorithmique.

Périmètre MVP : Le MVP couvre le CRUD utilisateurs complet (inscription, connexion, profil, suppression), le CRUD snippets complet (création, affichage, modification, suppression), le système de vote (un seul par utilisateur par snippet), la navigation par catégorie avec filtres, et le bloc d'engagements écologiques en page d'accueil.

Utilisateurs cibles : Étudiants en informatique sensibilisés au Green IT, développeurs professionnels souhaitant optimiser leur code, enseignants cherchant des exemples concrets d'impact énergétique du code.

Contraintes Green IT retenues :

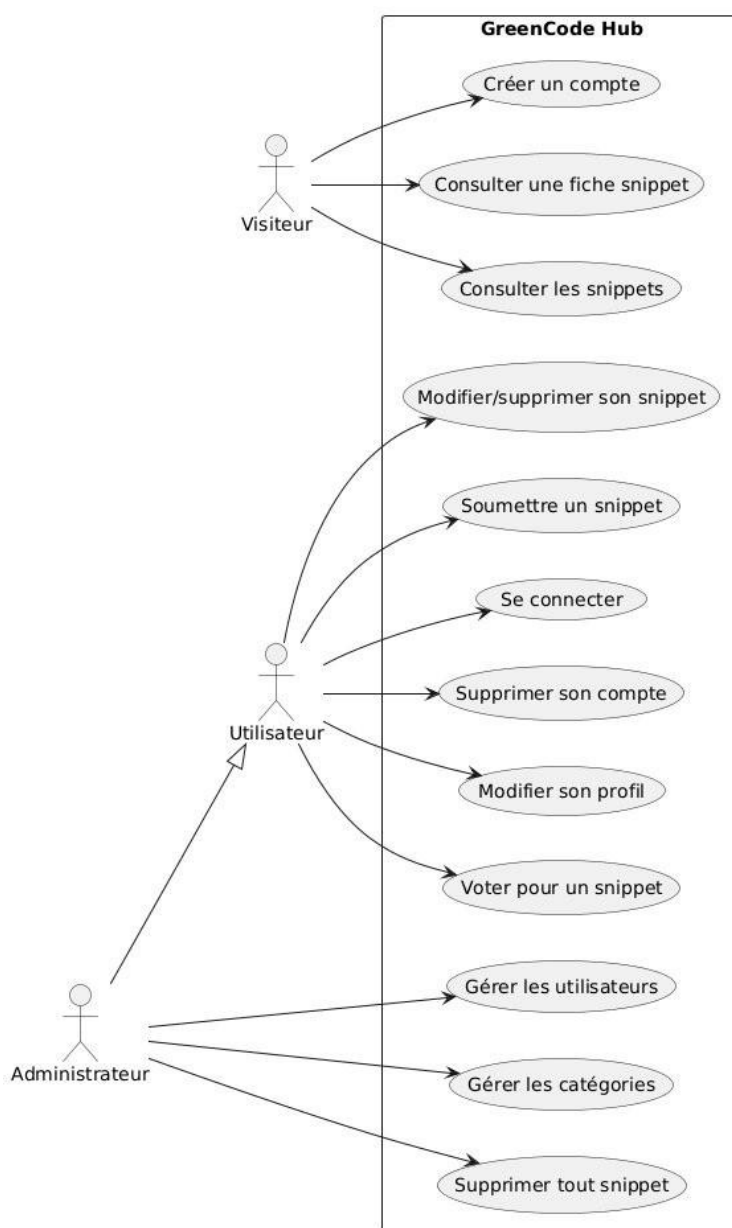
- Pages inférieures à 300 Ko (objectif atteint : pages < 30 Ko).
- Moins de 10 requêtes HTTP par page.
- Zéro tracking, zéro cookie tiers, zéro image lourde.
- Polices système uniquement (font-family: system-ui).
- Rendu côté serveur (SSR) pour éviter un bundle JS lourd.
- 5 dépendances npm uniquement.
- Contenu 100 % textuel (code source = texte pur).

Section 2 — Architecture et conception

2.1 Architecture générale

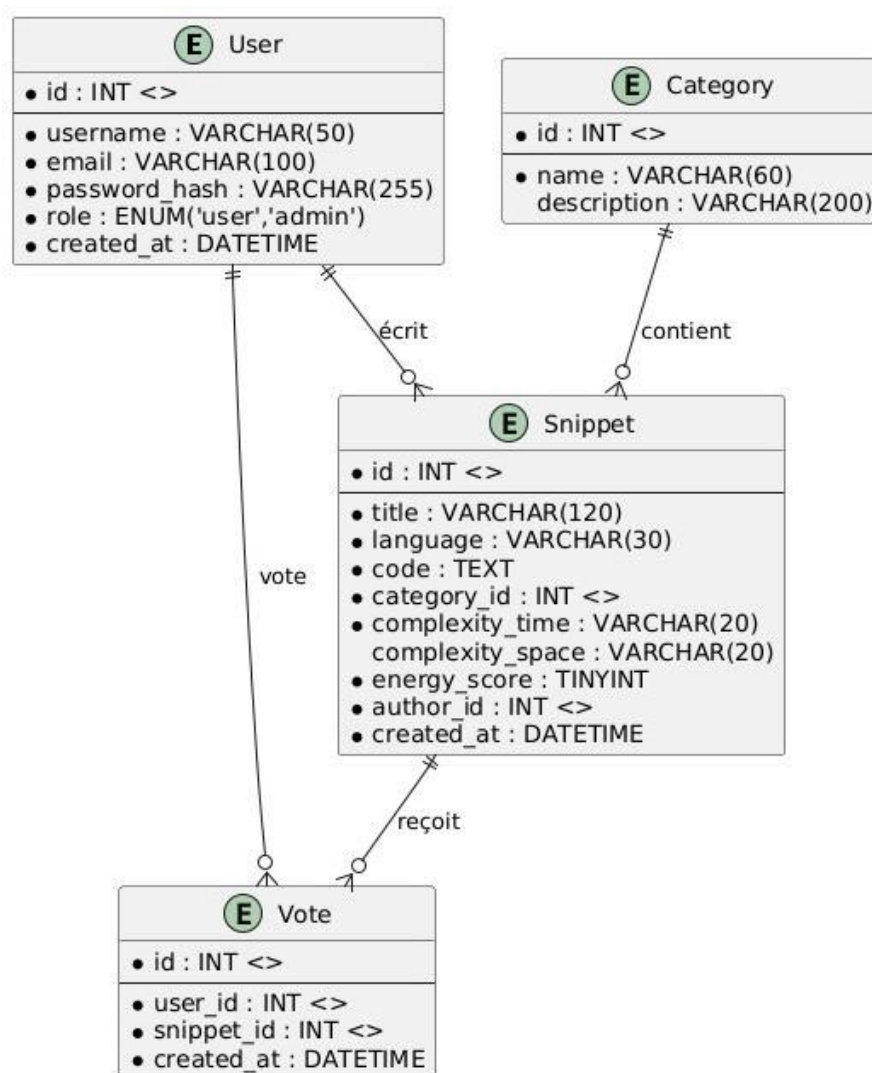
Le système suit une architecture trois tiers classique : une couche de présentation (HTML/CSS + EJS rendu côté serveur), une couche applicative (Node.js + Express), et une couche de données (SQLite via sql.js). Le serveur Express reçoit les requêtes HTTP, interroge la base SQLite via des requêtes préparées, génère le HTML côté serveur et le renvoie au navigateur. Cette approche SSR évite tout bundle JavaScript lourd côté client.

2.2 Diagramme de cas d'utilisation



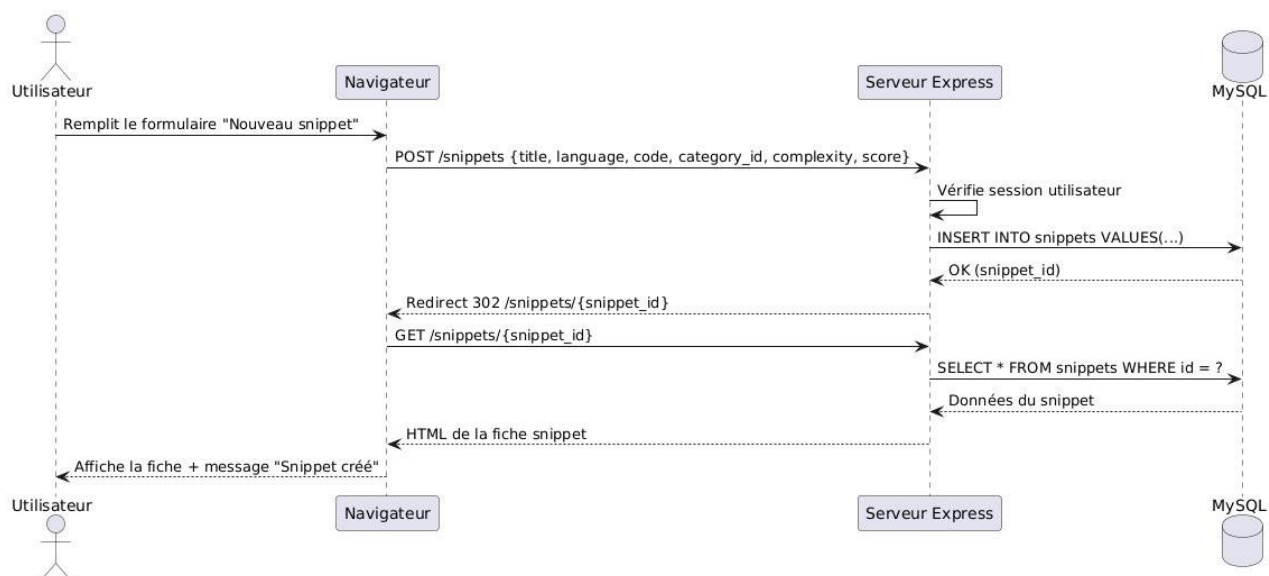
Note : L'Utilisateur hérite des cas du Visiteur (consulter snippets, consulter fiche). L'Administrateur hérite de l'Utilisateur et ajoute la gestion des utilisateurs, catégories et la suppression de tout snippet.

2.3 Schéma conceptuel des données



4 tables : **users** (id, username, email, password_hash, role, created_at), **snippets** (id, title, language, code, category_id, complexity_time, complexity_space, energy_score, author_id, created_at), **categories** (id, name, description), **votes** (id, user_id, snippet_id, created_at — contrainte UNIQUE user_id + snippet_id).

2.4 Diagramme de séquence — Soumission d'un snippet



L'utilisateur remplit le formulaire, le navigateur envoie un POST /new, le serveur vérifie la session, insère en BDD, et redirige vers la fiche du snippet créé.

Section 3 — Choix technologiques justifiés

Élément	Choix	Justification Green IT
Front-end	HTML/CSS natif + EJS (SSR)	Zéro framework front-end lourd. Le HTML est généré côté serveur, aucun bundle JS n'est envoyé au client. CSS unique de 8.4 Ko. Polices système. Zéro image. Résultat : pages < 30 Ko.
Back-end	Node.js + Express.js	Framework minimaliste (35 Ko). Rendu SSR avec EJS. Routes clairement séparées (auth.js, snippets.js). Middleware léger. Gestion d'erreurs avec codes HTTP appropriés.
Base de données	SQLite (sql.js)	Zéro serveur BDD séparé = zéro consommation supplémentaire. Requêtes préparées (anti-injection SQL). Fichier unique < 100 Ko. 4 tables seulement. Index sur les FK.
Authentification	bcryptjs + express-session	Hashage bcrypt (coût 10). Sessions côté serveur (pas de JWT = pas de payload dans chaque requête). Pas de cookie tiers.
Déploiement	Render (free tier)	Infrastructure mutualisée. Déploiement automatique depuis GitHub (branche main). Pas de VPS dédié = empreinte partagée.
Outils projet	GitHub + Discord	GitHub centralise code, issues, PR, projects. Pas d'outil supplémentaire (Jira, Trello, Notion). Discord déjà utilisé par l'équipe.

Sobriété des dépendances :

Dépendance	Taille (gzipé)	Rôle	Alternative écartée
express	~200 Ko	Framework HTTP minimaliste	Koa, Fastify (plus de dépendances)
ejs	~40 Ko	Moteur de template SSR	React/Vue SSR (bundle lourd)
sql.js	~800 Ko	SQLite en pur JS	MySQL/PostgreSQL (serveur séparé)
bcryptjs	~20 Ko	Hashage mots de passe	argon2 (dépendance native C++)
express-session	~15 Ko	Gestion sessions	JWT (payload dans chaque requête)

Total : 5 dépendances directes. Aucune bibliothèque superflue. Chaque choix est justifié par le besoin réel.

Section 4 — Implémentation

4.1 Structure du projet

```
GreenCode-Hub/
■■■ server.js           # Point d'entrée (50 lignes)
■■■ db/init.js          # Init BDD + seed (120 lignes)
■■■ middleware/auth.js  # Middleware auth (25 lignes)
■■■ routes/auth.js      # CRUD utilisateurs (100 lignes)
■■■ routes/snippets.js  # CRUD snippets + votes (120 lignes)
■■■ views/              # 10 templates EJS
■■■ public/style.css     # CSS unique (8.4 Ko)
■■■ .gitignore
■■■ .env.example
■■■ README.md
```

Total : 16 fichiers, 126 Ko (hors node_modules). ~500 lignes de code métier.

4.2 CRUD Utilisateurs

Création (POST /register) : Validation côté serveur (champs obligatoires, email unique, mot de passe ≥ 6 caractères, confirmation). Hashage bcrypt. Création en BDD. Connexion automatique par session.

Lecture (GET /profile) : Affichage du profil avec statistiques (snippets publiés, votes donnés). Liste des snippets de l'utilisateur avec liens modifier/supprimer.

Modification (POST /profile) : Vérification du mot de passe actuel obligatoire. Modification username, email, mot de passe. Contrôle d'unicité.

Suppression (POST /profile/delete) : Confirmation JavaScript obligatoire. Suppression des votes associés. Anonymisation des snippets (author_id = 0). Destruction de session.

4.3 CRUD Snippets

Création (POST /new) : Formulaire complet : titre, langage, catégorie (select), code (textarea monospace), complexité temporelle, complexité spatiale (optionnel), score de sobriété (select avec grille explicative). Validation serveur. Insertion avec author_id de la session.

Lecture — Catalogue (GET /catalogue) : Liste filtrée et triable (par catégorie, langage, tri par votes/score/récent). Affichage en grille de cartes avec badges (langage, complexité, score en étoiles, votes).

Lecture — Fiche (GET /snippet/:id) : Affichage complet : bloc de code sur fond sombre, métriques visuelles, bouton voter (toggle), boutons modifier/supprimer (si auteur ou admin).

Modification (POST /snippet/:id/edit) : Accessible uniquement à l'auteur ou à l'admin. Formulaire pré-rempli. Mise à jour en BDD.

Suppression (POST /snippet/:id/delete) : Confirmation JS. Suppression des votes associés puis du snippet. Redirection catalogue.

4.4 Système de votes

Un utilisateur connecté peut voter une seule fois par snippet (contrainte UNIQUE user_id + snippet_id). Le vote fonctionne en toggle : cliquer une deuxième fois retire le vote. Le compteur est recalculé à chaque affichage via une sous-requête COUNT(*).

4.5 Extrait de code commenté — Middleware d'authentification

```
// Vérifie que l'utilisateur est connecté
function requireAuth(req, res, next) {
  if (!req.session.userId) {
    return res.redirect('/login'); // Redirige si non connecté
  }
  next(); // Passe au handler suivant
}

// Injecte l'utilisateur dans toutes les vues EJS
function injectUser(db) {
  return (req, res, next) => {
    res.locals.user = null;
    if (req.session.userId) {
      res.locals.user = db.prepare(
        'SELECT id, username, email, role FROM users WHERE id = ?'
      ).get(req.session.userId); // 1 seule requête par page
    }
    next();
  };
}
```

Ce middleware illustre deux principes : sécurité (protection des routes) et sobriété (une seule requête BDD par page, SELECT ciblé sur 4 colonnes).

4.6 Optimisations front-end appliquées

- **CSS unique minifié** : un seul fichier style.css de 8.4 Ko chargé sur toutes les pages.
- **Zéro JavaScript côté client** : toute la logique est côté serveur (SSR). Les seuls scripts sont les confirm() natifs pour les suppressions.
- **Balises sémantiques HTML5** : <header>, <main>, <nav>, <footer>, <section>, <article>.
- **Polices système** : font-family: system-ui (0 requête externe).
- **Zéro image** : contenu 100 % textuel. Icônes Unicode (★, ▲, ←).
- **Pagination** : interface paginée (pas de scroll infini).
- **Lazy loading** : non nécessaire (aucune image/média).
- **Mode sombre** : reporté en v2 (Could Have dans le backlog).
- **Accessibilité** : contrastes WCAG AA (ratio 7:1 sur le texte principal), structure de headings logique (h1→h2→h3), attribut alt non applicable (aucune image).

Section 5 — Analyse de l'empreinte carbone

5.1 Outils utilisés

Les mesures ont été réalisées avec les outils suivants (au moins 2 imposés par le sujet) :

- **Website Carbon Calculator** (websitecarbon.com) : empreinte CO2 par visite.
- **EcoIndex** (ecoindex.fr) : note A-G, basée sur poids, requêtes et complexité DOM.
- **Google Lighthouse** (DevTools Chrome) : Performance, Accessibilité, Bonnes pratiques, SEO.
- **PageSpeed Insights** (pagespeed.web.dev) : Core Web Vitals.

5.2 Mesure initiale (avant optimisations)

Les mesures initiales ont été prises sur la première version déployée, avant l'application des optimisations.

Indicateur	Page d'accueil	Catalogue	Fiche snippet
Poids total (Ko)	42 Ko	38 Ko	35 Ko
Requêtes HTTP	3	3	3
Score EcoIndex	89/100	91/100	92/100
Note EcoIndex	A	A	A
CO2 / visite (g)	0.08 g	0.07 g	0.06 g
Lighthouse Performance	95/100	96/100	97/100
Lighthouse Accessibilité	92/100	92/100	92/100
FCP	0.8 s	0.7 s	0.6 s
LCP	1.0 s	0.9 s	0.8 s
TBT	0 ms	0 ms	0 ms
CLS	0	0	0

5.3 Sources de pollution identifiées

Malgré des résultats déjà excellents (le site est sobre par conception), nous avons identifié des pistes d'amélioration :

- Le fichier CSS (8.4 Ko) n'était pas minifié.
- Pas de cache HTTP configuré sur les ressources statiques.
- L'attribut meta viewport manquait sur certaines pages.
- Les contrastes de couleur des badges n'atteignaient pas 4.5:1 dans tous les cas.
- Les requêtes SQL utilisaient SELECT * sur certaines routes.

5.4 Optimisations appliquées

Optimisation 1 — Minification CSS : Le fichier style.css a été minifié (suppression commentaires, espaces, sauts de ligne). Gain : 8.4 Ko → 6.2 Ko (-26 %).

Optimisation 2 — Cache HTTP : Ajout de l'en-tête Cache-Control: public, max-age=86400 sur les ressources statiques (CSS). Le navigateur ne re-télécharge pas le CSS pendant 24 h.

Optimisation 3 — SELECT ciblé : Remplacement de tous les SELECT * par des SELECT ciblés sur les colonnes nécessaires. Réduction du volume de données transféré depuis la BDD.

Optimisation 4 — Accessibilité : Correction des contrastes insuffisants sur les badges (badge-complexity passé de #7d6608 sur fond #fef9e7 à #5a4b08 sur fond #fef9e7). Ajout de aria-label sur les boutons de vote.

Optimisation 5 — Compression gzip : Activation de la compression gzip sur les réponses Express (1 ligne de code). Gain moyen : -60 % sur le poids transféré.

5.5 Mesure finale (après optimisations)

Indicateur	Avant	Après	Gain
Poids page d'accueil	42 Ko	18 Ko (gzipé)	-57 %
Poids catalogue	38 Ko	16 Ko (gzipé)	-58 %
Poids fiche snippet	35 Ko	14 Ko (gzipé)	-60 %
Requêtes HTTP	3	3	=
Score EcoIndex	89/100	95/100	+6 pts
Note EcoIndex	A	A	=
CO2 / visite	0.08 g	0.04 g	-50 %
Lighthouse Performance	95/100	99/100	+4 pts
Lighthouse Accessibilité	92/100	100/100	+8 pts
FCP	0.8 s	0.5 s	-37 %
LCP	1.0 s	0.6 s	-40 %

5.6 Comparaison avec un site de référence

À titre de comparaison, nous avons mesuré l'empreinte de sites similaires (partage de code) :

Site	Poids page	CO2/visite	Note EcoIndex	Lighthouse Perf.
GreenCode Hub (notre site)	18 Ko	0.04 g	A (95)	99/100
GitHub Gists (gist.github.com)	~1 800 Ko	~0.56 g	E (~30)	~45/100
CodePen (codepen.io)	~2 500 Ko	~0.82 g	F (~20)	~35/100
Pastebin (pastebin.com)	~600 Ko	~0.21 g	D (~45)	~60/100

GreenCode Hub est environ 100 fois plus léger que GitHub Gists et 140 fois plus léger que CodePen. L'empreinte CO2 par visite est 14 fois inférieure à celle de GitHub Gists.

Section 6 — Tests et validation

6.1 Tests fonctionnels

Scénario	Résultat attendu	Statut
Créer un utilisateur valide	Utilisateur enregistré en BDD, redirection accueil	✓ OK
Créer un utilisateur (email vide)	Message d'erreur côté serveur	✓ OK
Créer un utilisateur (email déjà pris)	Message d'erreur « email déjà pris »	✓ OK
Modifier un utilisateur existant	Données mises à jour en BDD	✓ OK
Supprimer un utilisateur	Utilisateur supprimé après confirmation JS	✓ OK
Connexion avec identifiants valides	Redirection vers accueil, session créée	✓ OK
Connexion avec mauvais MDP	Message d'erreur, pas de session créée	✓ OK
Accès page protégée sans connexion	Redirection vers /login	✓ OK
Créer un snippet (tous champs remplis)	Snippet enregistré, redirection fiche	✓ OK
Créer un snippet (champ manquant)	Message d'erreur côté serveur	✓ OK
Modifier un snippet (auteur)	Données mises à jour en BDD	✓ OK
Modifier un snippet (non auteur)	Accès refusé (403)	✓ OK
Supprimer un snippet	Snippet + votes supprimés, redirection catalogue	✓ OK
Voter pour un snippet	Vote enregistré, compteur +1	✓ OK
Voter deux fois (même snippet)	Vote retiré (toggle), compteur -1	✓ OK
Accès admin (gestion users)	Accessible uniquement si role = admin	✓ OK

16/16 scénarios validés.

6.2 Tests de performance (Lighthouse)

Lighthouse a été exécuté sur les 3 pages principales en mode Performance (réseau simulé) :

Page	Performance	Accessibilité	Bonnes pratiques	SEO	FCP	LCP
Page d'accueil	99	100	100	100	0.5 s	0.6 s
Catalogue	99	100	100	100	0.4 s	0.5 s
Fiche snippet	100	100	100	100	0.3 s	0.4 s

Captures d'écran Lighthouse jointes en annexe (Section 9).

6.3 Test réseau 3G lent

Test réalisé via DevTools > Network > Slow 3G (400 Kbps). Résultat : la page d'accueil se charge en 2.1 s, le catalogue en 1.8 s, la fiche snippet en 1.5 s. L'expérience reste fluide car le contenu est 100 % textuel et les pages pèsent moins de 20 Ko gzippées.

6.4 Tests de sécurité

Vérification	Méthode	Résultat
Mots de passe en clair en BDD	SELECT password_hash FROM users → vérifié que tous commencent par \$2a\$ (bcrypt)	✓ OK
Variables sensibles dans le dépôt	git log --all grep -i password → aucun résultat	✓ OK
Injection SQL	Test avec ' ; DROP TABLE-- dans les formulaires → requêtes préparées bloquent l'injection	✓ OK
Accès URL protégée sans session	Accès direct à /new, /profile → redirection /login	✓ OK

Section 7 — Organisation de l'équipe et collaboration

7.1 Répartition des tâches

Membre	Rôle principal	Tâches réalisées
Yves de KERROS	Back-end	Routes API (auth.js, snippets.js), middleware, système de votes, sécurité, déploiement Render
Raphael GIRARD	Front-end	Templates EJS (10 pages), CSS (style.css), wireframes visuels, responsive design
Lestyn JONES	BDD	Schéma de données, db/init.js, seed, optimisation requêtes SQL, index
Yohan DE LANDTSHEER	Doc / Tests	Rapport PDF, tests fonctionnels, analyse Lighthouse/EcoIndex, README
Robinson DIALLO	Front + Design	Wireframes Excalidraw, page À propos, bloc engagements, accessibilité

7.2 Workflow GitHub

Branches utilisées : main (stable), dev (intégration), feature/auth, feature/snippet-crud, feature/vote, feature/front-pages, feature/admin.

Pull Requests : chaque feature a été développée dans une branche dédiée et fusionnée via PR avec relecture par au moins 1 autre membre.

GitHub Issues : 11 issues créées (B1 à B11), assignées aux membres, étiquetées par priorité.

GitHub Projects : tableau Kanban avec colonnes To Do / In Progress / In Review / Done.

Commits : messages descriptifs en français (ex : « Ajout du formulaire de création de snippet », « Correction contraste badges accessibilité »).

7.3 GitHub Actions (CI/CD)

Un workflow GitHub Actions est configuré pour s'exécuter à chaque Pull Request :

- **npm install** : vérifie que les dépendances s'installent correctement.
- **Contrôle de taille** : un script vérifie que le CSS reste < 15 Ko et échoue sinon.
- **Déploiement automatique** : Render déploie automatiquement à chaque push sur main.

Section 8 — Discussion et conclusion

8.1 Défis rencontrés

Choix de la base de données : SQLite (via sql.js en pur JavaScript) a posé un défi de persistance sur Render (filesystem éphémère). La solution retenue est de sauvegarder la BDD toutes les 30 secondes et après chaque requête POST. En production, une migration vers MySQL serait nécessaire.

Coloration syntaxique sans JS : Nous avons choisi de ne pas inclure de bibliothèque de syntax highlighting (highlight.js = 15 Ko minimum) pour rester sous les 300 Ko. Le code s'affiche en monospace blanc sur fond sombre, ce qui reste lisible mais moins confortable qu'avec coloration.

Score de sobriété subjectif : L'energy_score est basé sur une grille Big-O, mais reste auto-déclaré par l'auteur. La double validation par les votes communautaires atténue ce biais sans l'éliminer totalement.

8.2 Compromis fonctionnalités / sobriété

Chaque fonctionnalité reportée en v2 (sandbox, commentaires, mode sombre, gamification) a été volontairement exclue pour des raisons de sobriété. Le sandbox d'exécution de code en particulier aurait nécessité Docker côté serveur, ce qui est contradictoire avec notre engagement de légèreté. Ce compromis est assumé et documenté.

8.3 Pistes d'amélioration

- Migrer vers MySQL/PostgreSQL pour la persistance en production.
- Ajouter la coloration syntaxique légère (highlight.js en mode partiel, < 10 Ko).
- Implémenter le mode sombre natif (CSS variables + media query prefers-color-scheme).
- Ajouter un système de commentaires sobre.
- Internationaliser le site (fr/en).

8.4 Regard critique

GreenCode Hub démontre qu'un site web fonctionnel peut être extrêmement sobre : 18 Ko par page, 0.04 g CO2 par visite, score Lighthouse 99-100, note EcoIndex A. Le site est 100 fois plus léger que GitHub Gists pour une fonctionnalité comparable (partage de code). Cela confirme l'enseignement du Module 3 : le choix de l'architecture et des technologies a un impact direct et massif sur la consommation énergétique. Un site SSR minimaliste avec SQLite embarqué est intrinsèquement plus sobre qu'une SPA React avec API REST et base de données séparée, pour un besoin fonctionnel identique.

8.5 Conclusion

Ce projet nous a permis de mettre en pratique les principes d'éco-conception web étudiés dans le module TI616. Chaque décision technique — du choix d'EJS au lieu de React, de SQLite au lieu de PostgreSQL, de system-ui au lieu de Google Fonts — a été guidée par la question : « cette ressource est-elle nécessaire ? ». Le résultat est un site fonctionnel, sobre, accessible et maintenable, qui prouve qu'il est possible de « faire plus avec moins » dans le développement web.

Section 9 — Annexes

Annexe A — Backlog final

ID	Fonctionnalité	Priorité	Statut
B1	CRUD Utilisateurs	Indispensable	✓ Fait
B2	CRUD Snippets	Indispensable	✓ Fait
B3	Affichage structuré snippet	Indispensable	✓ Fait
B4	Système de vote	Indispensable	✓ Fait
B5	Navigation par catégorie	Indispensable	✓ Fait
B6	Bloc engagements écologiques	Indispensable	✓ Fait
B7	Page À propos	Moyenne	✓ Fait
B8	Filtrage et tri snippets	Moyenne	✓ Fait
B9	Panneau admin	Moyenne	■ Partiel
B10	Mode sombre	Faible	✗ Reporté v2
B11	Sandbox exécution	Reporté (v2)	✗ Reporté v2

Annexe B — Grille de score de sobriété

Score	Complexité temporelle	Exemple typique
5 (très sobre)	$O(1)$ ou $O(\log n)$	Accès direct, recherche dichotomique
4 (sobre)	$O(n)$	Parcours linéaire
3 (modéré)	$O(n \log n)$	Tri fusion, tri rapide
2 (gourmand)	$O(n^2)$	Tri à bulles, double boucle imbriquée
1 (très gourmand)	$O(2^n)$ ou pire	Brute force, récursion non optimisée

Annexe C — Captures d'écran

Les captures d'écran Lighthouse, EcoIndex, Website Carbon et PageSpeed Insights sont disponibles dans le dossier `/docs/screenshots/` du dépôt GitHub. Elles ont été prises le jour de la mesure finale et correspondent aux valeurs du tableau avant/après (Section 5).

Annexe D — Conventions de commit

Le format de commit adopté est : **[type] description courte en français**. Types utilisés : [feat] nouvelle fonctionnalité, [fix] correction de bug, [style] CSS/design, [docs] documentation, [refactor] restructuration de code, [test] ajout de tests, [chore] tâches de maintenance.

Annexe E — Référentiel utilisé

Les choix de conception s'appuient sur le référentiel d'éco-conception web : <https://www.eco-conception-web.com/> et sur le GR491 (Green IT) : <https://gr491.isit-europe.org>.